

🔍 Guide — Surveiller un run RAG/Ollama en direct (nvidia-smi, nvidia-smi, htop)

Fichier : Guide-Monitoring-RAG.md · **Dossier :** wiki (racine)
Branche : master (wiki) · **Auteur :** hprzeta · **MAJ :** 2026-07-25

🔗 **Rôle de cette page.** Comment observer *en direct* ce qui se passe pendant qu'un test RAG génératif (mathstral via Ollama) tourne sur PC1 (zeta-lab) : quels outils utiliser, quoi lire dans chaque affichage, et comment interpréter les chiffres. Rédigé à partir de la session du 25/07/2026 (premier test RAG après l'upgrade RAM 8 → 16 Go). Compagnon de [[Guide-Ollama-Pratique]] (config, benchmarks §5, incidents §7).

1. Pourquoi surveiller — et quoi surveiller

Un run RAG génératif sollicite **trois ressources** en même temps sur PC1 :

Ressource	Qui la consomme	Risque historique
VRAM (4 Go, GTX 960M)	mathstral chargé par Ollama	saturation → offload CPU
RAM (16 Go depuis le 25/07)	le débordement de mathstral + ChromaDB + embedding + Python	OOM (VS Code tué à 8 Go)
CPU (i7-7500U, 2c/4t)	la génération token par token (partie non-GPU)	goulot d'étranglement

💡 **Le vrai risque sur PC1, ce n'est pas la VRAM mais la RAM.** mathstral fait 4,1 Go, la VRAM n'en offre que 4 : Ollama charge ~3,3 Go sur GPU et **déborde le reste (~4,5 Go) en RAM** (offload partiel GPU/CPU). C'est ce débordement qui saturait les 8 Go et déclenchait l'OOM. Il faut donc surveiller **VRAM et RAM ensemble**.

2. Les trois outils — rôle de chacun

Les trois sont **complémentaires, pas redondants**. Chacun répond à une question différente.

Outil	Rôle / ce qu'il surveille	La question à laquelle il répond
nvidia-smi	État brut du GPU : driver, VRAM, util %, température, process GPU	« Le driver répond-il ? » (déecte le mismatch NVML)
nvidia-smi	GPU en dynamique : graphe temporel VRAM/util + tableau process avec HOST MEM	« Comment le modèle se répartit-il GPU ↔ CPU ? »
htop	RAM + CPU système : barre Mem/Swap, charge par cœur, load average	« Le système tient-il ? » (fin de l'OOM, goulot CPU)

Règle mnémotechnique : `nvidia-smi` = le GPU répond ? · `nvidia-smi` = où va le modèle ? · `htop` = le système survit ?

3. `nvidia-smi` — l'état brut du GPU

Dans un terminal **système** (hors VS Code), rafraîchi chaque seconde :

```
watch -n 1 nvidia-smi
```

Version compacte (VRAM + util + température seulement) :

```
watch -n 1 'nvidia-smi --query-gpu=memory.used,memory.total,utilization.gpu,temperature.gpu --format=csv,noheader,nounits'
```

Ce qu'il faut lire

- **En-tête :** `NVIDIA-SMI 580.173.02 · Driver 580.173.02 · CUDA 13.0` → confirme que le bon driver est chargé. Si `NVML: Driver/library version mismatch`, le driver kernel et userspace divergent (un `apt upgrade` a installé un nouveau driver sans reboot) → **reboot requis**.
- **Ligne GPU :** `Perf P0` = pleine puissance (carte réveillée), `Memory-Usage 514MiB / 4096MiB`, `GPU-Util 39%`, température.
- **Tableau Processes :** colonne **Type**. `G` = Graphics (affichage : Xorg, firefox, gnome-shell), `C` = **Compute** (calcul). **mathstral apparaît en C** avec ~3,3 Go de GPU Memory *seulement pendant la génération*.

⚠ **Au repos, aucun process `C` ollama n'apparaît, c'est normal.** Ollama décharge le modèle de la VRAM après inactivité (`OLLAMA_KEEP_ALIVE`). Le process `ollama serve` (le serveur) reste, mais `ollama runner` (le modèle chargé) n'existe que pendant/juste après une requête. VRAM ~0,5 Go au repos = mathstral endormi, pas un problème.

4. `nvidia-smi` — le GPU en dynamique (le plus parlant)

```
pip install nvidia-smi # ou : sudo apt install nvidia-smi
nvidia-smi
```

Ce qu'il faut lire

- **En-tête** : PCIe GEN3 x4 · TX 636 MiB/s (gros TX = modèle en train d'être poussé en VRAM), fréquences GPU/MEM hautes, température, GPU 44%, MEM 3.770Gi / 4.000Gi .
- **Graphe (historique)** :
- **ligne mem** % monte en plateau ~95 % et **y reste** → VRAM remplie, modèle chargé.
- **ligne GPU util** % en **dents de scie** (0 → 100 → 40...) → **génération token par token** (chaque pic = calcul d'un batch, chaque creux = transfert CPU ↔ GPU). Signature typique de l'**offload partiel**.
- **Tableau process** — la ligne clé :

```
PID 54467 Compute GPU MEM 3267MiB (80%) HOST MEM 4460MiB ollama runner --model ../mathstral
```

- **Type Compute** → c'est bien du calcul.
 - **GPU MEM 3267 MiB** → mathstral occupe 3,3 Go de VRAM (80 % de la carte).
 - **HOST MEM 4460 MiB** → et **4,46 Go en RAM en plus** ← la preuve de l'offload partiel.
- ⚠ Le modèle ne tient pas entièrement dans les 4 Go de VRAM (mathstral = 4,1 Go) → Ollama en met 3,3 Go sur GPU et déborde 4,46 Go sur CPU/RAM. **À 8 Go, ce débordement tuait le système.**

5. `htop` — RAM + CPU système (la preuve de survie)

```
htop
# F4 pour filtrer : taper « oll » → n'affiche que les process ollama
```

Ce qu'il faut lire

- **Barre Mem** : 4.90G / 15.5G → **~32 % utilisés, ~10 Go libres. Barre Swap : 0K / 16G** → **zéro swap. C'est LA preuve que l'upgrade RAM a réglé l'OOM.** À 8 Go, on était au plafond ici → OOM → VS Code tué.
- **En-tête CPU** : 4 cœurs à ~44-56 %, Load average 6.16 . **Un load de 6 sur 4 threads = plus de travail en attente que de cœurs.** → **Le goulot d'étranglement est le CPU, pas la RAM.**
- **Colonne RES** : chaque `ollama runner` affiche 4460M / 28.1% .

⚠ **Piège à éviter : ~10 lignes à 4460M ≠ 10 × 4,46 Go.** Ce sont les **threads du même process** (PID principal + ses fils). `htop` affiche la mémoire du process entier pour chaque thread. La vraie conso = **4,46 Go au total, une seule fois** (confirmé par la barre Mem à 4,9 Go). Les `TIME+` le montrent : seul le thread principal a du CPU cumulé, les autres ~0.

6. Bilan des trois vues (test du 25/07, Q1 cold-start)

Métrique	Valeur observée	Source	Verdict
Driver	580.173.02 (CUDA 13.0)	nvidia-smi	NVML OK ✓
VRAM mathstral	3 267 MiB (80 %)	nvidia-smi	modèle sur GPU
VRAM totale carte	3,77 / 4,0 Go (~94 %)	nvidia-smi	quasi plein
RAM host (runner)	4 460 MiB	nvidia-smi / htop	offload CPU
RAM système	4,9 / 15,5 Go (~32 %)	htop	fin OOM ✓
Swap	0 K / 16 Go	htop	aucun ✓
GPU-util	40-100 % oscillant	nvidia-smi	génération active
CPU load average	6.16 (sur 4 threads)	htop	goulot = CPU
Temp GPU	66 °C	nvidia-smi / nvidia-smi	normal
PCIe TX	636 MiB/s	nvidia-smi	chargement modèle

Les deux conclusions

1. **L'OOM est résolu.** RAM 4,9/15,5 Go, swap 0 → les 16 Go donnent ~10 Go de marge. À 8 Go, OOM ici.
2. **Le nouveau (et seul) goulot est le CPU.** Load 6.16 sur 4 threads. Pour accélérer la génération RAG : viser un **modèle ≤ VRAM** (phi3:mini 2,2 Go tient entièrement dans les 4 Go → plus d'offload, donc plus rapide) plutôt qu'ajouter de la RAM. (À noter en « Questions ouvertes ».)

7. Script `monitor_rag.sh` — les 3 vues d'un coup

Un script `tmux` (`~/projet_zeta/scripts/monitor_rag.sh`) ouvre une session `zeta-monitor` en 3 volets : GPU/VRAM (gauche, refresh 1 s), RAM système (haut-droite), process ollama (bas-droite). Statut cyan = PC1 (convention couleurs projet). Trois gardes en tête : refuse de tourner si `tmux` manque, si `nvidia-smi` est absent, ou s'il **répond en erreur** (mismatch NVML) — cette dernière garde protège du problème qui a nécessité le reboot du 25/07.

```
cd ~/projet_zeta
./scripts/monitor_rag.sh          # lance (ou réattache) la session
# Ctrl+b puis d → détacher (sans tuer)
tmux kill-session -t zeta-monitor # fermer
```

8. Deux pièges rencontrés le 25/07 (à connaître)

8.1 — torch abandonne Maxwell (embedding GPU cassé)

Une mise à jour de PyTorch (`torch 2.11.0+cu130`) a **abandonné le support de l'architecture Maxwell (compute capability 5.0)** de la GTX 960M. Conséquence : `sentence-transformers` **plante** en tentant d'utiliser le GPU pour calculer l'embedding de la question.

Contournement — forcer l'embedding sur CPU (l'embedding est léger, le CPU suffit) :

```
CUDA_VISIBLE_DEVICES="" python scripts/rag_query.py
```

🔔 **mathstral n'est pas affecté** : Ollama utilise son propre moteur (llama.cpp), pas PyTorch. La génération garde donc le GPU ; seul l'embedding bascule sur CPU. Alternative durable : figer torch à une version qui supporte encore Maxwell.

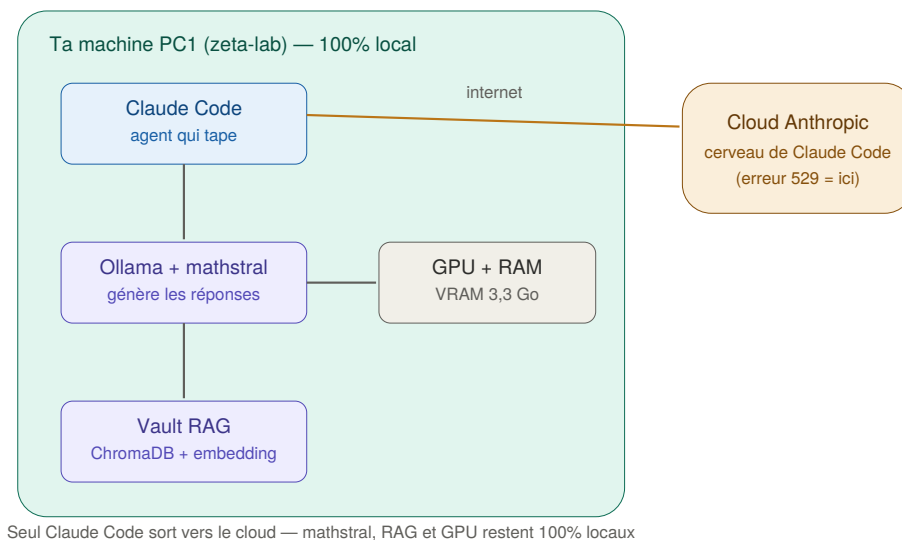
8.2 — l'erreur 529 vient du cloud, pas de la machine

Pendant le test, Claude Code a affiché `API Error: 529 Overloaded`. **Ce n'est ni la RAM, ni le GPU, ni le réseau local** : c'est le **cloud Anthropic** qui sature (vérifiable sur `status.claude.com`). Claude Code (l'agent qui tape les commandes) dépend du cloud ; **mathstral et le RAG, eux, sont 100 % locaux** et continuent de fonctionner cloud ou pas. Un `python scripts/rag_query.py` lancé à la main tourne même si tout le cloud est en panne — c'est le point d'indépendance de l'Objectif 2.

Voir le schéma de flux `flux_local_vs_cloud.svg` (dossier `docs/images/`).

Annexe — Schémas

Flux local vs cloud — seul Claude Code sort vers le cloud ; mathstral et le RAG restent 100% locaux :



Les trois outils de monitoring et la question à laquelle chacun répond :

Un run RAG sollicite 3 ressources — 3 outils pour les voir



Verdict du test du 25/07 (après upgrade 8 → 16 Go)

1. OOM résolu : RAM 4,9 / 15,5 Go, swap 0 → ~10 Go de marge (à 8 Go : OOM ici).
2. Nouveau goulot = CPU (load 6,16 sur 4 threads), pas la RAM. Piste : modèle ≤ VRAM.

Voir aussi

- [\[\[Guide-Ollama-Pratique\]\]](#) — config GPU, benchmarks \$5, historique incidents \$7
- [\[\[Guide-RAG-BrainVault-Debutant\]\]](#) — comprendre et surveiller la partie IA
- [\[\[STACK\]\]](#) — matériel (RAM 16 Go depuis 25/07), roadmap Objectif 2
- [scripts/monitor_rag.sh](#) — les 3 vues d'un coup (tmux)
- [scripts/rag_query.py](#) — le test RAG industrialisé (lançable en local, sans cloud)